Memory-Pressure Forecasting and Growth-Based Warnings: An Empirical Study

Abstract. Forecasting rising memory pressure can support monitoring, but accurate forecasts do not by themselves identify memory leaks. We compare persistence, linear trend, restricted ARIMA and a small LSTM on one Antarex fault-injection recording and three Westermo industrial recordings, using chronological splits and 60- or 300-second horizons. Forecast error and growth-warning performance are evaluated separately. In the original Antarex run, LSTM reduces RMSE from ARIMA's 1.516 to 1.216 percentage points of RAM. Its warning rule, however, recognizes 34 of 35 injected events against trend-only's 35, with timestamp recall 0.466 versus 0.859. A matched study of five random seeds and two LSTM input configurations adds 40 runs. Extra features improve mean RMSE on only two of four recordings. Multivariate Antarex warning F1 is 0.571 ± 0.031, below trend-only's 0.830, and false alarm starts range from 43 to 137 against 86. Analyzing the saved scores shows that the original missed event occurred simply because the forecast-growth threshold was not reached. Because the Westermo dataset lacks verified leak ground truth, it can only support forecasting evaluations. Furthermore, while local replay and inference benchmarks confirm prototype viability, we have not yet validated live runtime performance or lead-time warnings prior to memory exhaustion.

Keywords: Memory pressure · Time-series forecasting · LSTM · Warning evaluation · Operating-system telemetry

1 Introduction

An operating system can report growing memory use without identifying its cause. A larger workload, a cache, or an application retaining allocations can produce similar aggregate measurements. A forecast of memory availability is therefore a resource estimate, not proof of a memory leak. This distinction matters when a forecasting model is used to trigger operational warnings.

We study whether a more accurate short-horizon forecast gives more useful warnings on recorded telemetry. The target is the memory-pressure proxy

$$y_t = 1 - \frac{\text{available memory}_t}{\text{total physical memory}_t}. \tag{1}$$

It measures the fraction of RAM unavailable for immediate use. Models predict y_{t+h} using observations through time t. Warning rules then assess recent growth, optionally requiring the forecast to indicate continued growth. These two tasks are evaluated separately.

The study uses controlled Antarex fault injections and three unlabeled Westermo industrial test-system recordings. Its contribution is empirical: (1) a reproducible comparison of forecast error and warning coverage under the same temporal evaluation protocol; (2) a matched five-seed comparison of memory-only and multivariate LSTMs; and (3) an explanation of a missed injected event using the saved decision scores. We also measure the local replay and inference prototype. Our focus is not on introducing a novel neural network architecture or pinpointing allocation-level root causes. Likewise, evaluating early warning alerts prior to memory exhaustion falls outside the scope of this work.

2 Related Work and Research Gap

Software-aging literature links resource degradation and workload dynamics to system failures, ranging from measurement-based exhaustion estimates [32] and web-server stress tests [8] to broad surveys on rejuvenation and online failure prediction [6,26]. Yet these foundational studies do not treat every rising RAM curve as a confirmed leak; as Zhao et al. showed through fault-injection experiments [36], analyzing exhaustion requires ground-truth failure outcomes.

Analytical conclusions are also bound by observation granularity. Heap-level insights typically require dynamic instrumentation like Valgrind [21] or adaptive or garbage-collector profiling [10,15]. Other frameworks inspect embedded Linux runtime metrics [3], compare software versions differentially [20], or integrate full diagnosis and mitigation into production via systems like RESIN [18]. Because our aggregate telemetry captures neither heap allocations nor paired software builds, we refrain from making root-cause diagnostic claims.

For macro-level time series, prior work has detected leak patterns in utilization logs via LBR and Precog variants [14] and compared ARIMA-style Box–Jenkins models with LSTMs on Windows telemetry [1]. In this domain, statistical methods and simple baselines remain critical yardsticks [4,12]. Even though LSTMs provide expressive sequence modeling [11], comparative architecture studies and competitive linear baselines demonstrate that complex neural models do not inherently outperform simpler alternatives [7,34].

Hybrid ARIMA–neural forecasting [35], software-fault ARIMA–LSTM forecasting [27], and hLSTM-Aging [2] use different model combinations. We evaluate separate predictors and do not reproduce their architectures or compare scores from different tasks.

Table 1 summarizes the closest methodological contrasts. Its final column describes the boundary relevant to this study, rather than asserting that an earlier method is generally inadequate.

Assumptions and benchmark quality affect anomaly-detection results [5,33,28]. Forecast errors and warning scores assess different outcomes [13,25,31]. Chronological holdouts and training-only scaling make the information available at each prediction explicit [30,16].

Table 1. Selected earlier studies and the evidence they provide.

Study	Observation and purpose	Boundary for the present comparison
Jindal et al. [14]	Utilization histories; LBR/Precog leak-pattern methods	Their labels and algorithms differ from our injected-event and forecast-warning protocol.
Assi et al. [1]	Windows counters; ARIMA/LSTM forecast errors	Forecast-error ranking alone does not answer our timestamp/event warning question.
Zhao et al. [36]	Injected leaks; accelerated failure study	Failure outcomes are available there; our audited block has no exhaustion outcome.
Lou et al. [18]	Production service with detection, diagnosis and mitigation	Requires evidence beyond our aggregate replay and growth warnings.
Battisti et al. [2]	Hybrid neural model for software-aging forecasts	Different architecture and target setting; its scores cannot rank our warning rules.
Kim et al. [17]	Evaluation of time-series anomaly detection	Warns about point adjustment; we retain timestamp scores and separate event counts.

Research gap. When analyzing these traces under a single temporal protocol, does a lower forecasting error genuinely improve growth-warning coverage, cut down on false alarms, or speed up recognition? Answering this requires a controlled evaluation. We tackle this problem by pairing matched forecast origins with injected-event labels, running trials across repeated random seeds, and conducting a memory-only ablation. As for the unlabeled workload data, its role is strictly limited to comparing forecast performance rather than supporting leak-detection claims.

3 Data and Experimental Protocol

Figure 1 summarizes the archived workflow. The forecast and warning branches are evaluated separately: labels calibrate and score the warning rules, but never enter the predictors. The live sampler is a separate exploratory trend-only path, described in Algorithm 1.

3.1 Dataset Choice and Forecast Horizons

We chose the Antarex dataset because it pairs authentic machine telemetry with logged, controlled fault injections [22,24,23]. Specifically, our experiments draw on the fully recovered CPU-MEM_mono block, which spans roughly 12 days and captures 228 injected leak episodes, filtering down to only the metric streams required by our pipeline. While we verified the integrity of this nested archive against its recorded byte size and CRC checksum, we did not download the full outer archive or check its published MD5 hash. Consequently, the traces analyzed here should be viewed as a targeted subset rather than a full validation of the complete Antarex release.

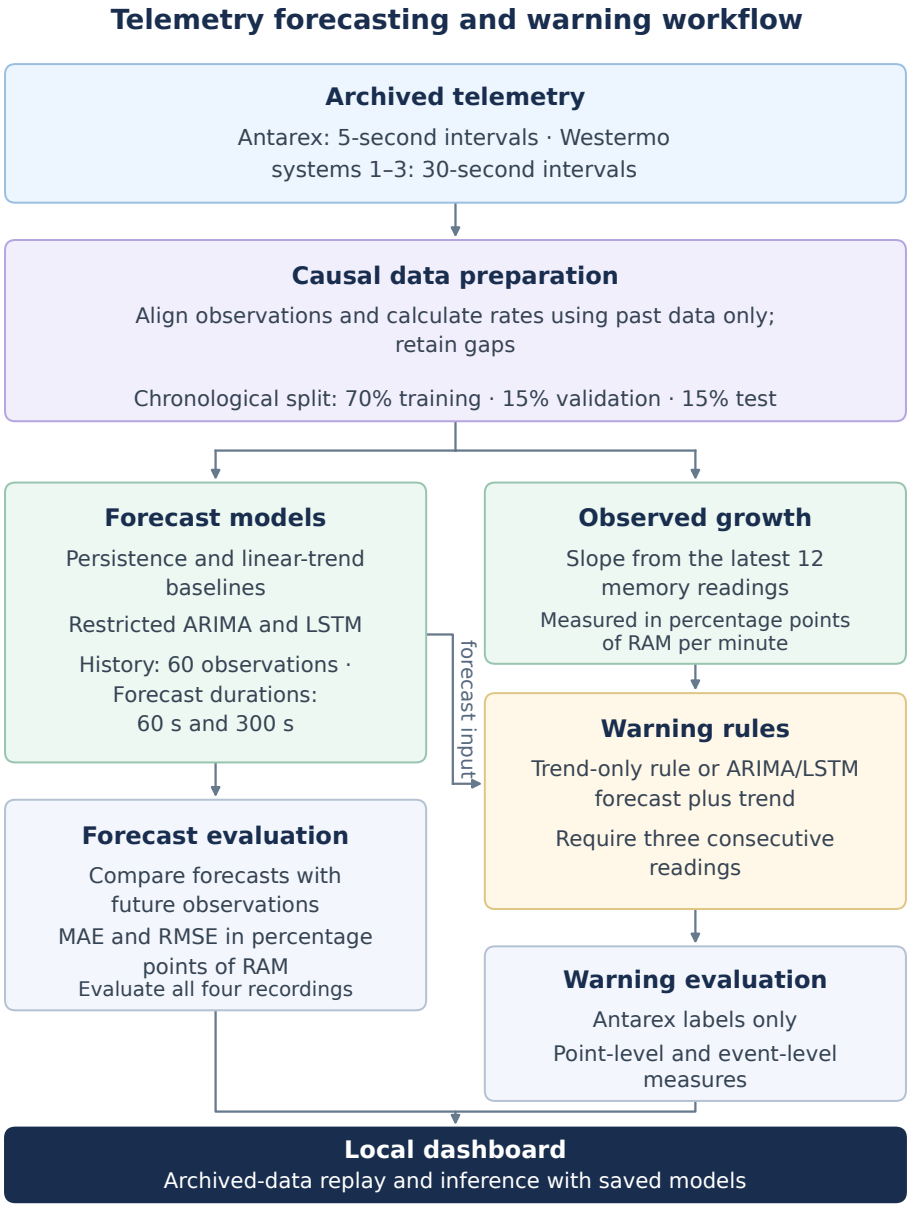


Fig. 1. Architecture of the archived experiment. Horizons are 60 s for Antarex and 300 s for Westermo. Only ARIMA and LSTM forecasts assist the growth rules; persistence and linear trend are forecast baselines. Warning evaluation separates timestamp precision, recall and F1 from event misses, false alarm starts and recognition delay. Westermo supports forecast scores and descriptive alarm counts, not labeled warning scores. The original ARIMA warning rule was disabled after validation. pp: percentage points of RAM.

To complement this with an industrial workload, we also incorporated the Westermo dataset, which tracks 19 test systems sampled twice a minute over a one-month period [29]. After downloading all 19 CSV files and cross-checking them against the repository metadata, we selected the first three systems (Systems 1–3) in numerical order for this pilot study, modeling each system independently. Because Westermo lacks ground-truth memory-leak annotations, we restrict its use to evaluating forecast accuracy and tallying descriptive alarm counts. Both datasets are distributed under the CC BY 4.0 license, and we maintain full attribution to the original authors.

We align Antarex to a 5-second grid while retaining Westermo's native 30-second sampling interval, setting default forecast horizons to 12 steps (60 s) and 10 steps (300 s), respectively. These horizons keep the step-ahead count near ten observations while respecting each source's sampling rate, serving as pragmatic pilot defaults rather than validated optimal warning lead times. Likewise, because a 60-step lookback window spans 295 s in Antarex versus 1,770 s in Westermo, error magnitudes across the two datasets should not be interpreted as a controlled comparison over equal wall-clock time.

Table 2. Recording scope and eligible samples. A: Antarex; W1–W3: Westermo systems. Train is the eligible count before LSTM training stride.

Recording	Grid rows	Train	Validation	Test	Inputs
A	207361	145081	31032	31032	7
W1	86401	59691	12891	12891	8
W2	86401	60412	12891	12745	7
W3	86401	60412	12891	12891	8

3.2 Causal Preparation and Feature Availability

For Antarex, the most recent source observation at or before each grid time is used, with a maximum age of 2.1 s. Differences between counter readings are used to calculate page-fault, swap, and disk rates based on the actual elapsed time. A rate is considered invalid if a counter resets, the time interval is nonpositive, or the interval exceeds 7.2 s. CPU features are calculated using non-overlapping tick categories. For Westermo, the published CPU and disk rates are used directly without further differentiation. Missing values and gaps are not interpolated.

The selected Antarex inputs include memory pressure, anonymous and committed memory, busy and I/O-wait CPU fractions, and total and major page-fault rates. Swap and monitored disk streams remain constant in the training partition and are therefore excluded. Westermo uses memory pressure, user/system/I/O-wait CPU rates, disk read/write rates, one-minute load, and swap use. For system 2, the constant swap input is removed. Westermo does not contain page-fault fields. Inputs with a training standard deviation of at most 10^{-12} are treated as constant. Their removal prevents undefined standardization and avoids adding uninformative inputs. However, this does not mean

that disk or swap is unimportant for other workloads. The feature ablation described below evaluates the retained feature set as a whole rather than examining the causal effect of each individual feature.

Each recording is divided according to time into 70% training, 15% validation, and 15% test sets. These boundaries are established before the windows are constructed. The training means and population standard deviations are then used to scale the subsequent data. A window is retained only if both its history and target fall within the same valid, contiguous segment and partition. The first Antarex row in each partition is excluded to prevent rate calculations from depending on observations from the preceding partition. Westermo systems 1 and 2 contain 652 and 8 missing grid rows, respectively. The memory-only ablation uses exactly the same coverage mask as the multivariate model, ensuring that differences in sample availability cannot explain the results.

For Antarex, the injection start and end times define half-open labeled intervals $[s, e)$. These intervals are checked against the supplied fault labels. IDs, timestamps, commands, and fault labels are not used as predictor inputs. The labels establish the controlled injection task, but no process-allocation measurements are available to confirm naturally occurring memory leaks. Additionally, a non-leak label may coincide with another injected fault, so it does not necessarily indicate that the system is healthy.

3.3 Predictors and Repeated Runs

Persistence repeats the latest memory pressure. Linear trend extends a least-squares line fitted to 12, 30 or 60 recent values; validation RMSE selects the window. Restricted ARIMA uses orders $(1, 1, 0)$, $(3, 1, 0)$ and $(5, 1, 0)$ without drift. Parameters are estimated by Yule–Walker on the longest contiguous training segment, and validation RMSE selects the order. This limited comparison does not implement a full Box–Jenkins search. Forecasts are clipped to $[0, 1]$ for scoring.

Both LSTM variants use one unidirectional layer with 32 hidden units and a linear output from its final hidden state. The recurrent state resets for each window. Training uses standardized-target mean squared error, Adam at 0.001, batch size 64 and gradient clipping at 1.0. Training retains every twelfth eligible Antarex origin and every fourth Westermo origin; validation/test use all eligible origins. The best validation-RMSE checkpoint is selected from at most 20 epochs, with patience four. There is no dropout, bidirectional layer or hybrid ARIMA stage.

The original September Colab experiment used seed 42. The revision trains both feature variants from scratch with seeds 7, 21, 42, 84 and 123 on the same local CPU runtime: 40 runs in total. All checkpoints are frozen before the new test scores are computed. Every seed is reported, with mean and sample standard deviation (SD). This is variation from training randomness on fixed data, not uncertainty across new machines. Memory-only LSTM matches the baselines' input variable; model fitting and training sample use still differ. Fixed hidden width yields 4513 parameters for memory-only, versus 5281 or 5409 for seven or eight inputs. No new labeled recording is added.

4 Warning Procedure and Evaluation

4.1 Growth Rules and Threshold Calibration

Let g_t be the least-squares slope over the latest 12 memory-pressure observations, expressed in percentage points (pp) of RAM per minute. For forecast $f_m(t)$ with horizon h in seconds, define

$$q_m(t) = \frac{100[f_m(t) - y_t]}{h/60}, \qquad s_m(t) = \min\{g_t, q_m(t)\}. \tag{2}$$

Trend-only uses $s(t) = g_t$. Forecast-assisted warnings require both observed and forecast growth. A warning becomes active on the third consecutive eligible score at or above its threshold; one failure clears it. Gaps and split boundaries reset the count. Three readings suppress isolated excursions but add 10 s of confirmation at Antarex cadence and 60 s at Westermo cadence. This was a fixed engineering choice, not an optimized duration or a statistical-significance test. Mann–Kendall testing and autocorrelation corrections are alternative future comparisons [19,9].

To tune the warning rules, each variant independently evaluates thresholds from the set $\{0.01, 0.025, 0.05, 0.1, 0.2, 0.4, 0.8, 1.6, 3.2\}$ pp/min, plus a disabled state. We optimize for pointwise validation F1 on Antarex. If multiple thresholds tie, we favor the one that generates fewer false-positive timestamps; if a tie still persists, we pick the higher threshold, treating the disabled option as the highest rank of all. In our original run, this procedure selected 0.8 pp/min for the trend-only baseline and 0.4 pp/min for the LSTM-assisted variant. For ARIMA, however, every candidate threshold yielded zero post-confirmation warnings on the validation split, causing the tie-breaking logic to select the disabled option. This edge case is specific to our chosen no-drift ARIMA specification and rule pairing—it does not imply that ARIMA is fundamentally incapable of driving warning systems. Across repeated runs, each newly trained LSTM recalibrates its threshold exclusively on the validation split, while we keep the original run's outputs archived separately.

Algorithm 1 illustrates how we isolate offline model evaluation from the live monitoring loop. When running live, the system samples OS metrics every five seconds through `psutil`, derives the same memory proxy and an exploratory observed-growth rule, and exports the telemetry—deliberately avoiding running the archived LSTM on mismatched Windows feature streams.

4.2 What Each Measure Means

MAE is the average absolute difference between predicted and observed future pressure. RMSE squares the errors before averaging and taking the square root, giving large mistakes greater weight [13]. Both are reported in pp of total RAM. An RMSE of 1.5 pp describes forecast deviation on this scale; it is neither 98.5% detection accuracy nor a bound on every error.

At each eligible origin, TP means a warning during a recorded leak, FP a warning outside one, and FN an active leak without a warning. Precision = TP/(TP + FP) asks how many warned timestamps are leak-active. Recall = TP/(TP + FN) asks how much

Algorithm 1 Telemetry forecasting and causal warning generation

Input: archived telemetry or a live OS sampler; history window of 60 samples; trend window of 12 samples

if the data come from an archived experiment **then**
    Align the recorded observations in time and compute rates; keep gaps and note each value's source
    Split the data chronologically, fit the scalers on training rows only, and build the valid input windows
    Train the predictors on the training set and keep the checkpoints with the lowest validation RMSE
    Choose the warning thresholds from the validation labels, then fix them
else
    Start the five-second sampler and set an exploratory trend threshold
end if
for each new observation at time t **do**
    Read the available telemetry and compute y_t using Eq. 1
    After a gap, clear the history and the count, then wait until enough history has built up
    Compute the observed slope g_t over the last 12 readings
    if the inputs are compatible with an archived model **then**
        Scale the history using the saved training statistics and predict $f_m(t)$
        Set $s_t = g_t$ for trend-only rules, or compute s_t from Eq. 2 for model-assisted rules
    else
        Set $s_t = g_t$; no substitute model inputs are made up
    end if
    Update the count of consecutive qualifying steps and raise a warning once the count reaches 3
    Store the forecast, the score, the warning, and any change in alarm state
end for
Evaluate the stored forecasts and decisions only where suitable outcomes are available

labeled leak time is covered. $F1 = 2TP/(2TP + FP + FN)$ balances these two aspects. Zero-denominator ratios remain undefined. Imbalance makes these measures more informative than overall accuracy alone [25].

An alarm start is an inactive-to-active transition, including activation after a gap. False alarm starts per non-leak hour (FA/h) divide starts outside injected-leak intervals by observed non-leak exposure: eligible non-leak origins times cadence. Persistent false warnings are also reported as false-positive minutes. These are distinct costs.

Event recognition requires a new alarm start inside a fully covered injected interval, with a covered sample immediately before it. A prior alarm does not count again. Event recall is recognized events divided by eligible events; detection delay is first alarm time minus injection start. Medians and 90th percentiles describe detected events only, so missed-event counts accompany them. Detecting one timestamp does not make every timestamp in that interval correct [31,17]. These delays measure recognition after injection begins, not warning lead time before exhaustion.

5 Results and Interpretation

5.1 Forecast Accuracy and the Baseline Comparison

Table 3 preserves the original run. On Antarex, LSTM RMSE was 1.216 pp against ARIMA's 1.516 pp, a 19.8% reduction, while MAE changed only from 0.363 to 0.358 pp. The larger RMSE improvement indicates a stronger effect on large errors than on average absolute error. Across Westermo, the original LSTM reduced RMSE but ARIMA had lower MAE: no method wins under every loss.

The selected Antarex ARIMA$(3, 1, 0)$ coefficients are approximately -0.011, -0.010 and -0.011. With no drift, forecast increments quickly become small, making its predictions close to persistence; their RMSEs differ by only 0.000196 pp. Linear extrapolation performs worse in all four recordings, consistent with transient slopes not continuing through the forecast horizon. Figure 2 shows the fixed original example interval; full-test scores, rather than that interval, support the comparisons.

Table 3. Original test forecast errors: MAE / RMSE, in percentage points (pp) of total RAM. A uses 60 s; W1–W3 use 300 s. LSTM is the single Colab seed-42 run.

Model	A	W1	W2	W3
Persistence	0.361 / 1.517	0.094 / 0.363	0.109 / 0.317	0.240 / 0.715
Linear trend	0.377 / 1.744	0.145 / 0.376	0.144 / 0.348	0.329 / 0.818
ARIMA	0.363 / 1.516	0.093 / 0.318	0.105 / 0.291	0.230 / 0.658
Original LSTM	0.358 / 1.216	0.099 / 0.296	0.117 / 0.258	0.251 / 0.608

5.2 Random Seeds and the Memory-Only Ablation

The new CPU runs use Python 3.12.14, PyTorch 2.8.0+cpu and two Torch threads. Table 4 keeps them separate from Colab. On Antarex, multivariate mean RMSE is 1.204 pp versus memory-only 1.221 pp. Paired multivariate RMSE is lower in four of five Antarex seeds and all five W1 seeds, but only one W2 seed and no W3 seed. On W2 and W3, extra inputs also increase MAE in every paired seed. Additional telemetry therefore does not consistently improve this fixed model and training budget. The experiment does not isolate which feature helps or harms.

Both LSTM variants have lower mean RMSE than restricted ARIMA on each recording, but all their mean MAEs exceed ARIMA's. Training minimizes squared error, which weights large deviations strongly; lower RMSE with worse MAE is therefore possible. Seed SD describes optimization variability on these recordings, not a confidence interval for other systems. No additional labeled dataset or independent holdout was introduced.

5.3 Warnings, Missed Events and Recognition Delay

There are 1933 leak-active origins among 31,032 Antarex test origins and 35 fully covered injected events. Table 5 shows that the original LSTM warning improves precision and reduces false alarm starts from 86 to 42, but covers less leak-active time and

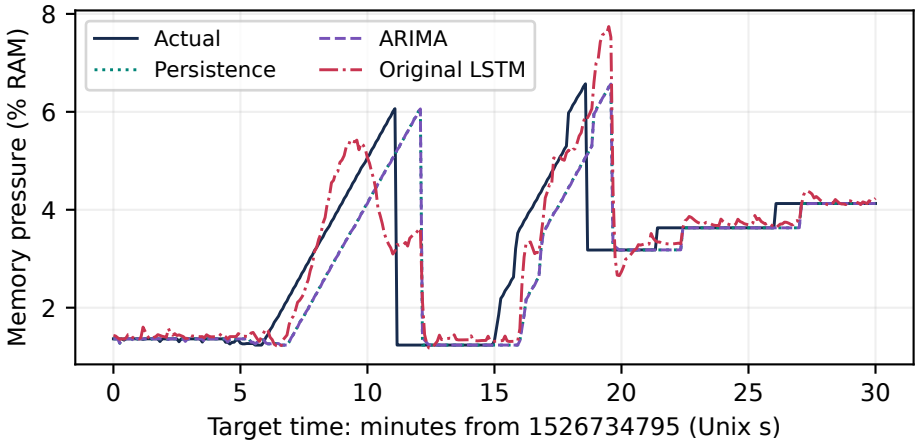


Fig. 2. Original Antarex 60-second-ahead rolling forecasts, aligned by target time. Predictions shown at a time were issued 60 s earlier. Persistence and ARIMA nearly overlap. This is the first fixed 30-minute test interval, not a best-case selection.

Table 4. Five-seed LSTM test errors: mean ± sample SD, in pp. Mem: memory-only; Multi: retained multivariate inputs. All five seeds are included.

Recording	MAE (Mem)	MAE (Multi)	RMSE (Mem)	RMSE (Multi)
A	0.378 ± 0.018	0.365 ± 0.024	1.221 ± 0.015	1.204 ± 0.020
W1	0.099 ± 0.008	0.103 ± 0.008	0.300 ± 0.001	0.297 ± 0.002
W2	0.110 ± 0.005	0.121 ± 0.004	0.260 ± 0.003	0.263 ± 0.003
W3	0.242 ± 0.011	0.278 ± 0.026	0.606 ± 0.005	0.618 ± 0.009

misses one event. False alarm rates are 2.128/h for trend-only and 1.039/h for LSTM; false-positive exposure is 34.17 and 13.50 min respectively. Their detected-event 90th-percentile delays are 56.0 and 45.7 s. The disabled ARIMA rule is equivalent to always-off and misses all events.

The difference follows from the warning objective. Regression rewards accurate future levels over all origins, whereas the assisted rule demands both observed and forecast growth above a threshold for three readings. A model can predict the level closely while predicting insufficient continued growth. The additional condition suppresses some false alarms and some true warnings; minimizing forecast RMSE does not optimize event recall or warning F1.

Table 5. Original Antarex test warnings. P/R: timestamp precision/recall; FA: false alarm starts; missed: events missed out of 35; delay: median seconds after injection among detected events only. A dash means undefined.

Rule	P	R	F1	FA	Missed	Delay (s)
Trend only	0.802	0.859	0.830	86	0	30
LSTM + trend	0.848	0.466	0.601	42	1	31
ARIMA (off)	–	0.000	0.000	0	35	–

Event 1534, a low-intensity injection starting at 22:00:14 UTC on 19 May 2018, demonstrates this mechanism (Fig. 3). During its 236 s and 47 eligible origins, forecast growth peaks at 0.216 pp/min, below the 0.4 threshold. Thus no reading can satisfy the forecast gate, irrespective of the three-reading requirement. Observed growth reaches 0.938 pp/min; trend-only warns after 56 s. Memory pressure spans approximately 3.20–7.17%. The recorded scores explain the decision failure, not its allocation-level cause.

The seed study weakens a general claim of fewer false alarms (Table 6). Multivariate precision is 0.758 ± 0.112 and recall 0.461 ± 0.010; memory-only gives 0.648 ± 0.071 and 0.482 ± 0.021. Multivariate false starts range from 43 to 137, compared with the unchanged trend count of 86; memory-only ranges from 126 to 168. Each variant misses zero to two events depending on seed. Trend-only F1 remains 0.830, above every new LSTM-assisted run. The original precision/false-alarm advantage is therefore not stable across all repetitions. Validation selects thresholds separately for each run; forecast training variability and threshold decisions both influence these results.

Table 6. Repeated LSTM-assisted Antarex warnings. F1 and FA/h are mean ± SD; missed is the range out of 35 events. Delay summarizes the five detected-event medians, not pooled events.

Inputs	F1	FA/h	Missed	Delay (s)
Multivariate	0.571 ± 0.031	2.014 ± 1.052	0–2	26.6 ± 3.4
Memory-only	0.551 ± 0.028	3.405 ± 0.449	0–2	30.4 ± 2.7

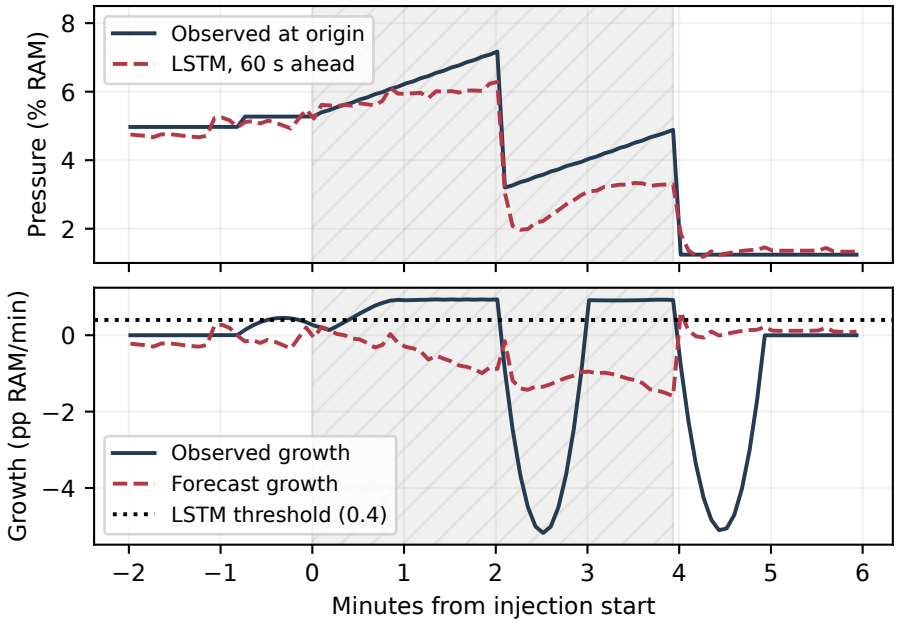


Fig. 3. Original missed injected event 1534. Shading marks the recorded injection. Top: current pressure and the forecast issued at the same origin (its target is 60 s later). Bottom: observed and forecast growth; predicted growth remains below the LSTM gate throughout the injection. pp: percentage points of RAM.

5.4 Prototype Checks and Measurement Limits

All 40 implementation tests passed, including loopback HTTP endpoints. Startup verified 76 source files covering 69,559 test origins. In a separate in-process benchmark on Windows with 8 logical CPUs and 7.8 GiB RAM, each recording received 100 replay/JSON requests and 100 warm model requests. Median replay times ranged from 7.05 to 21.84 ms; median warm inference ranged from 0.45 to 0.56 ms (95th percentiles 0.69–0.96 ms). All 404 first/warm inferences matched archived predictions within 10^{-6} pressure fraction. The first request, including library/model loading, took 5.34 s. Process resident memory increased from 25.4 to 402.3 MiB; this includes replay data, libraries and four models, not isolated monitoring overhead.

These measurements exclude HTTP transport and browser rendering. The live collector returned a Windows performance-counter error before any samples were recorded. No live timing, CPU-overhead or labeled live-detection result is reported. Longer live trials, missing-counter resilience and controlled overhead measurements remain unvalidated. The optional live mode uses exploratory trend warnings, not the archived Linux LSTM.

6 Limitations, Reproducibility and Conclusion

The extended study is still based on a single labelled Antarex recording, controlled injections, overlapping time windows and reused holdouts. Some of the earlier test forecasts were examined during development, before the original warning protocol was finalized. Repeating the experiments with five different seeds helps measure variation in optimization, but it does not eliminate the influence of development choices. The multivariate and memory-only comparison accounts for history, origins and training settings, although the input dimensions and number of parameters also change. Therefore, the improvement cannot be linked to any one feature alone. The selected horizons, thresholds and confirmation length also need to be validated independently.

Antarex does not reach the illustrative 80% pressure level within the audited block. The highest observed pressure is approximately 42.24%. Since no exhaustion or crash was observed, the study cannot claim that the system can warn before memory runs out. Establishing this would require labelled trials with defined resource limits, measured depletion times and warning lead times, including cases where the warning fails to appear. Westermo cannot be used to establish real leak detection because its leak labels are unavailable. Further work should include more labelled recordings, process-level allocation evidence, multivariate statistical baselines evaluated under matched conditions, independent temporal holdouts and longer live trials in a compatible environment.

Reproducibility. The project preserves the preprocessing steps, baseline, LSTM and warning code, four executable notebooks, model configurations, weights, origin indices, checksums and scoring scripts. This revision also introduces a fixed seed and feature protocol, forecasts and warning outcomes for each run, a missed-event analysis and a prototype benchmark. To reproduce the results, the verified input package is used to run the seed study, after which the saved predictions and warning scores are checked independently. The repository remains private, so the materials are not presented as publicly

accessible. A masked supplementary code package is available to support author review, while access to the raw and derived data packages must be arranged separately. The public dataset sources are cited above. However, a public code release or a reviewer-access arrangement is needed before the work can be described as openly reproducible.

Conclusion. Lower forecasting error and improved warning performance should be treated as separate results. The LSTM achieved a lower RMSE than the restricted ARIMA model, whereas the trend-only approach covered more labelled time and obtained a higher warning F1 score. The improvement in LSTM precision and false alarms observed in the original study was not consistent across all seeds. Additional features reduced the mean RMSE in only two of the four recordings, while the missed-event scores revealed a forecast-growth threshold that the missed event did not meet. These results help distinguish forecasting accuracy from the performance of operational warnings. However, they do not establish production-level leak diagnosis or prove that the system can warn before memory exhaustion. Independent labelled trials, process-level measurements and longer live evaluations are required next.

References

1. Assi, M.J., Fahad, A.A., Al-Sarray, B.: Time series forecast modeling for the Windows operating system performance using Box-Jenkins and LSTM models. International Journal of Nonlinear Analysis and Applications **13**(2), 2121–2131 (2022). https://doi.org/10.22075/ijnaa.2022.26974.3505
2. Battisti, F., Silva, A., Pereira, L., Carvalho, T., Araujo, J., Choi, E., et al.: hLSTM-Aging: A Hybrid LSTM Model for Software Aging Forecast. Applied Sciences **12**(13), 6412 (2022). https://doi.org/10.3390/app12136412
3. Beneder, R., Glatz, B., Horauer, M., Rauscher, T.: Memory leak detection runtime-service for embedded Linux devices. In: Proceedings of the 2014 IEEE Emerging Technology and Factory Automation (ETFA). pp. 1–6. IEEE (2014). https://doi.org/10.1109/ETFA.2014.7005223
4. Box, G.E.P., Jenkins, G.M., Reinsel, G.C.: Time Series Analysis: Forecasting and Control. Wiley, 4th edn. (2008). https://doi.org/10.1002/9781118619193
5. Chandola, V., Banerjee, A., Kumar, V.: Anomaly detection: A survey. ACM Computing Surveys **41**(3), 1–58 (2009). https://doi.org/10.1145/1541880.1541882
6. Cotroneo, D., Natella, R., Pietrantuono, R., Russo, S.: A survey of software aging and rejuvenation studies. ACM Journal on Emerging Technologies in Computing Systems **10**(1), 1–34 (2014). https://doi.org/10.1145/2539117
7. Greff, K., Srivastava, R.K., Koutnik, J., Steunebrink, B.R., Schmidhuber, J.: LSTM: A Search Space Odyssey. IEEE Transactions on Neural Networks and Learning Systems **28**(10), 2222–2232 (2017). https://doi.org/10.1109/TNNLS.2016.2582924
8. Grottke, M., Li, L., Vaidyanathan, K., Trivedi, K.S.: Analysis of Software Aging in a Web Server. IEEE Transactions on Reliability **55**(3), 411–420 (2006). https://doi.org/10.1109/TR.2006.879609
9. Hamed, K.H., Ramachandra Rao, A.: A modified Mann-Kendall trend test for autocorrelated data. Journal of Hydrology **204**(1–4), 182–196 (1998). https://doi.org/10.1016/S0022-1694(97)00125-X

10. Hauswirth, M., Chilimbi, T.M.: Low-overhead memory leak detection using adaptive statistical profiling. In: Proceedings of the 11th International Conference on Architectural Support for Programming Languages and Operating Systems. pp. 156–164. ACM (2004). https://doi.org/10.1145/1024393.1024412
11. Hochreiter, S., Schmidhuber, J.: Long Short-Term Memory. Neural Computation **9**(8), 1735–1780 (1997). https://doi.org/10.1162/neco.1997.9.8.1735
12. Hyndman, R.J., Athanasopoulos, G.: Forecasting: Principles and Practice. OTexts, 3rd edn. (2021), https://otexts.com/fpp3/
13. Hyndman, R.J., Koehler, A.B.: Another look at measures of forecast accuracy. International Journal of Forecasting **22**(4), 679–688 (2006). https://doi.org/10.1016/j.ijforecast.2006.03.001
14. Jindal, A., Staab, P., Kulkarni, P., Cardoso, J., Gerndt, M., Podolskiy, V.: Memory Leak Detection Algorithms in the Cloud-based Infrastructure. arXiv preprint arXiv:2106.08938 (2021). https://doi.org/10.48550/arXiv.2106.08938
15. Jump, M., McKinley, K.S.: Cork: dynamic memory leak detection for garbage-collected languages. In: Proceedings of the 34th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages. pp. 31–38. ACM (2007). https://doi.org/10.1145/1190216.1190224
16. Kapoor, S., Narayanan, A.: Leakage and the reproducibility crisis in machine-learning-based science. Patterns **4**(9), 100804 (2023). https://doi.org/10.1016/j.patter.2023.100804
17. Kim, S., Choi, K., Choi, H.S., Lee, B., Yoon, S.: Towards a Rigorous Evaluation of Time-Series Anomaly Detection. Proceedings of the AAAI Conference on Artificial Intelligence **36**(7), 7194–7201 (2022). https://doi.org/10.1609/aaai.v36i7.20680
18. Lou, C., Chen, C., Huang, P., Dang, Y., Qin, S., Yang, X., et al.: RESIN: A Holistic Service for Dealing with Memory Leaks in Production Cloud Infrastructure. In: 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI). pp. 109–125. USENIX Association (2022), https://www.usenix.org/conference/osdi22/presentation/lou-resin
19. Mann, H.B.: Nonparametric Tests Against Trend. Econometrica **13**(3), 245–259 (1945). https://doi.org/10.2307/1907187
20. Matias, R., Andrzejak, A., Machida, F., Elias, D., Trivedi, K.: A Systematic Differential Analysis for Fast and Robust Detection of Software Aging. In: 2014 IEEE 33rd International Symposium on Reliable Distributed Systems. pp. 311–320. IEEE (2014). https://doi.org/10.1109/SRDS.2014.38
21. Nethercote, N., Seward, J.: Valgrind: a framework for heavyweight dynamic binary instrumentation. In: Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation. pp. 89–100. ACM (2007). https://doi.org/10.1145/1250734.1250746
22. Netti, A., Kiziltan, Z., Babaoglu, O., Sirbu, A., Bartolini, A., Borghesi, A.: Antarex HPC Fault Dataset. Zenodo, version 1.0 (2018). https://doi.org/10.5281/zenodo.2553224
23. Netti, A., Kiziltan, Z., Babaoglu, O., Sirbu, A., Bartolini, A., Borghesi, A.: Online Fault Classification in HPC Systems through Machine Learning. arXiv preprint arXiv:1810.11208 (2018). https://doi.org/10.48550/arXiv.1810.11208, revised 2019
24. Netti, A., Kiziltan, Z., Babaoglu, O., Sîrbu, A., Bartolini, A., Borghesi, A.: FINJ: A Fault Injection Tool for HPC Systems. In: Euro-Par 2018: Parallel Processing Workshops. Lecture Notes in Computer Science, vol. 11339, pp. 800–812. Springer International Publishing (2019). https://doi.org/10.1007/978-3-030-10549-5_62
25. Saito, T., Rehmsmeier, M.: The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. PLOS ONE **10**(3), e0118432 (2015). https://doi.org/10.1371/journal.pone.0118432
26. Salfner, F., Lenk, M., Malek, M.: A survey of online failure prediction methods. ACM Computing Surveys **42**(3), 1–42 (2010). https://doi.org/10.1145/1670679.1670680

27. Samal, U., Kumar, A.: Improving software reliability: a hybrid ARIMA-LSTM approach for fault prediction. International Journal of System Assurance Engineering and Management **16**(5), 1757–1769 (2025). https://doi.org/10.1007/s13198-025-02743-2
28. Schmidl, S., Wenig, P., Papenbrock, T.: Anomaly detection in time series: a comprehensive evaluation. Proceedings of the VLDB Endowment **15**(9), 1779–1797 (2022). https://doi.org/10.14778/3538598.3538602
29. Strandberg, P.E., Marklund, Y.: The Westermo test system performance data set. arXiv preprint arXiv:2311.14510 (2023). https://doi.org/10.48550/arXiv.2311.14510
30. Tashman, L.J.: Out-of-sample tests of forecasting accuracy: an analysis and review. International Journal of Forecasting **16**(4), 437–450 (2000). https://doi.org/10.1016/S0169-2070(00)00065-0
31. Tatbul, N., Lee, T.J., Zdonik, S., Alam, M., Gottschlich, J.: Precision and Recall for Time Series. In: Advances in Neural Information Processing Systems. vol. 31 (2018), https://proceedings.neurips.cc/paper/2018/hash/8f468c873a32bb0619eaeb2050ba45d1-Abstract.html
32. Vaidyanathan, K., Trivedi, K.S.: A measurement-based model for estimation of resource exhaustion in operational software systems. In: International Symposium on Software Reliability Engineering. pp. 84–93. IEEE (1999), https://scholars.duke.edu/publication/755326
33. Wu, R., Keogh, E.: Current Time Series Anomaly Detection Benchmarks are Flawed and are Creating the Illusion of Progress. IEEE Transactions on Knowledge and Data Engineering **35**(3), 2421–2429 (2023). https://doi.org/10.1109/TKDE.2021.3112126
34. Zeng, A., Chen, M., Zhang, L., Xu, Q.: Are Transformers Effective for Time Series Forecasting? Proceedings of the AAAI Conference on Artificial Intelligence **37**(9), 11121–11128 (2023). https://doi.org/10.1609/aaai.v37i9.26317
35. Zhang, G.P.: Time series forecasting using a hybrid ARIMA and neural network model. Neurocomputing **50**, 159–175 (2003). https://doi.org/10.1016/S0925-2312(01)00702-0
36. Zhao, J., Jin, Y., Trivedi, K.S., Matias Jr., R.: Injecting Memory Leaks to Accelerate Software Failures. In: 2011 IEEE 22nd International Symposium on Software Reliability Engineering. pp. 260–269. IEEE (2011). https://doi.org/10.1109/ISSRE.2011.24